QUANT DEV / C++ / SYSTEMS / CASE STUDY

EXCHANGE LAB

CASE STUDY

06

Exchange Lab asks which speed optimisations keep a matching engine exactly correct.

PROPOSED RESEARCH / NOT YET VERIFIED

THE BUSINESS CONTEXT

A small trading technology team maintains an order matching engine and wants it faster. Each change risks altering which orders match or in what sequence. Benchmarks are easily distorted by the machine and the method. This case study builds a deterministic engine on synthetic order flow and measures carefully.

WHO IS INVOLVED AND WHAT THEY NEED

01	**Engineering lead**	Needs speed gains that do not change matching outcomes.
02	**Risk reviewer**	Needs accounting and ordering rules proven under hostile input.
03	**Performance engineer**	Needs benchmark methods others can repeat and challenge.

SCOPE AND HONEST LIMITS

- ⚠ Synthetic flow does not show readiness for a production exchange.
- ⚠ Microbenchmarks depend on hardware and method and may not transfer.
- ⚠ The engine is single threaded and excludes networking and clearing.
- ⚠ No performance figure is claimed until measured and reproduced.

CURRENT WORKFLOW AND FAILURE POINTS

06

PROPOSED RESEARCH / NOT YET VERIFIED

HOW THE WORK RUNS TODAY

1 TODAY
Write the engine
A first order book is written with few tests beyond simple examples.

2 TODAY
Try an optimisation
A data structure or layout change is made to reduce delay.

3 TODAY
Run a quick benchmark
A single run on a busy laptop produces an average time.

4 TODAY
Compare by eye
Output is checked on a few cases and looks about right.

5 TODAY
Ship the change
The faster version replaces the old one with little evidence.

6 TODAY
Debug later
A rare ordering bug appears with no journal to reproduce it.

WHERE IT BREAKS

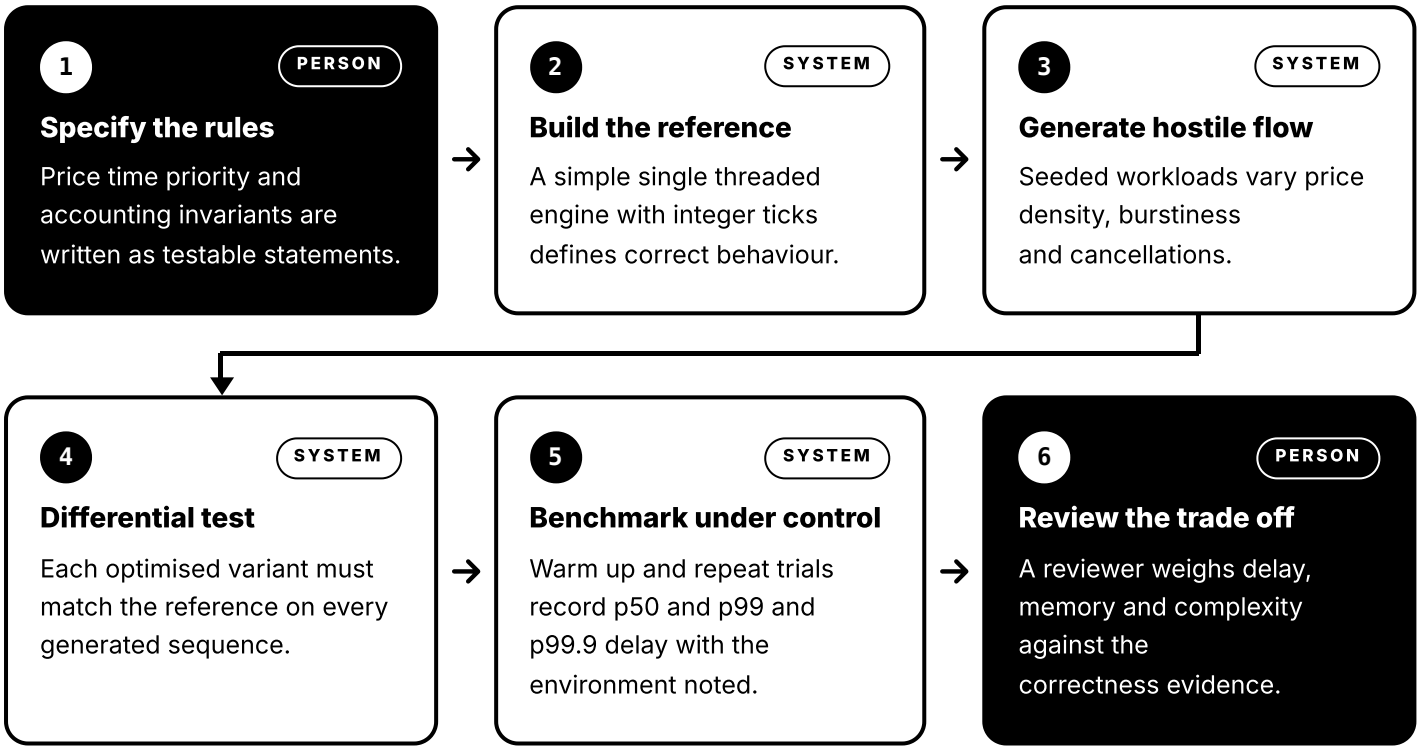
Hidden behaviour change	Optimisation alters matching in rare cases that tests do not reach.
Misleading benchmarks	Averages hide tail delay and noisy machines distort every result.
No reproduction path	Failures cannot be replayed without a complete event record.
Weak adversarial coverage	Tests use tidy flow while real flow bursts and cancels heavily.

REDESIGNED WORKFLOW AND DECISION AUTHORITY

06

PROPOSED RESEARCH / NOT YET VERIFIED

THE REDESIGNED WORKFLOW



SYSTEM STEP PERSON DECISION

WHERE AUTHORITY STAYS WITH A PERSON

Accept an optimised variant	ENGINEERING LEAD	A faster engine is only acceptable with equal behaviour on the full suite.
Define matching and risk rules	RISK REVIEWER	Rules set what correct means so a person owns them.
Publish benchmark claims	PERFORMANCE ENGINEER	Each claim needs its method and environment checked before release.

ARCHITECTURE, DATA AND CONTROLS

06

PROPOSED RESEARCH / NOT YET VERIFIED

SYSTEM ARCHITECTURE

1	**Order generator**	Seeded synthetic flow with adversarial patterns and recorded sequences.
2	**Risk and sequencing**	Pre trade checks and a single ordered event stream.
3	**Matching engine**	Reference and optimised books with integer prices and stable order identifiers.
4	**Journal and replay**	Complete event record that rebuilds state and verifies determinism.
5	**Benchmark harness**	Controlled runs, repeat trials and recorded compiler and machine details.

CORE DATA ENTITIES

- **Order**
- **Price level**
- **Trade**
- **Journal event**
- **Risk limit**
- **Benchmark run**

RISKS AND THE CONTROL FOR EACH

Optimised variant changes matching →	Differential tests compare every variant with the reference on all sequences.
Non deterministic replay →	Replay the journal repeatedly and compare final state byte for byte.
Distorted benchmark result →	Fix workloads and warm up then repeat trials and record the environment.
Accounting drift →	Check quantity and cash conservation after every generated sequence.

EVALUATION AND DELIVERY PLAN

06

PROPOSED RESEARCH / NOT YET VERIFIED

HOW SUCCESS IS MEASURED

Differential test agreement	Share of generated sequences where variant and reference outputs match exactly.
Invariant violations	Count of broken ordering or accounting rules across fuzzing runs.
Latency percentiles	p50 and p99 and p99.9 event processing delay under fixed workloads.
Memory footprint	Resident memory per resting order at stated book depths.
Replay determinism	Identical final state across repeated replays of the same journal.

TEST PLAN

1. **Property based fuzzing of add and cancel and amend and partial fill sequences.**
2. **Differential comparison of each optimised variant against the reference engine.**
3. **Replay of recorded journals confirms identical trades and final book state.**
4. **Repeated benchmark trials with warm up and a documented environment.**

DELIVERY PLAN

01 ALIGN	02 UNDERSTAND	03 DESIGN	04 DELIVER	05 LAUNCH
Agree the matching rules and what counts as a valid speed claim.	Study realistic order flow shapes and known sources of benchmark noise.	Define the reference model and the event format and the test protocol.	Build variants and the harness then run the correctness and latency campaigns.	Publish the code and raw measurements and the method with clear limits.
DECISION Which rules define correct	**DECISION** Which workloads to include	**DECISION** Approve the reference and protocol	**DECISION** Accept the correctness evidence	**DECISION** Which claims are fit to publish

DELIVERABLES

- Reference engine and optimised variants
- Differential and property test suite
- Reproducible benchmark harness
- Journal and replay tool
- Trade off report with raw measurements